Python Decorators and Functions

What is a Decorator in Python?

A decorator is a feature in Python that allows you to **add extra functionality** to an existing function **without modifying its original code**.

Think of a decorator as a **wrapper** that:

- takes a function,
- adds additional behavior to it,
- and returns a new enhanced function.

Extension of Function Without Decorators

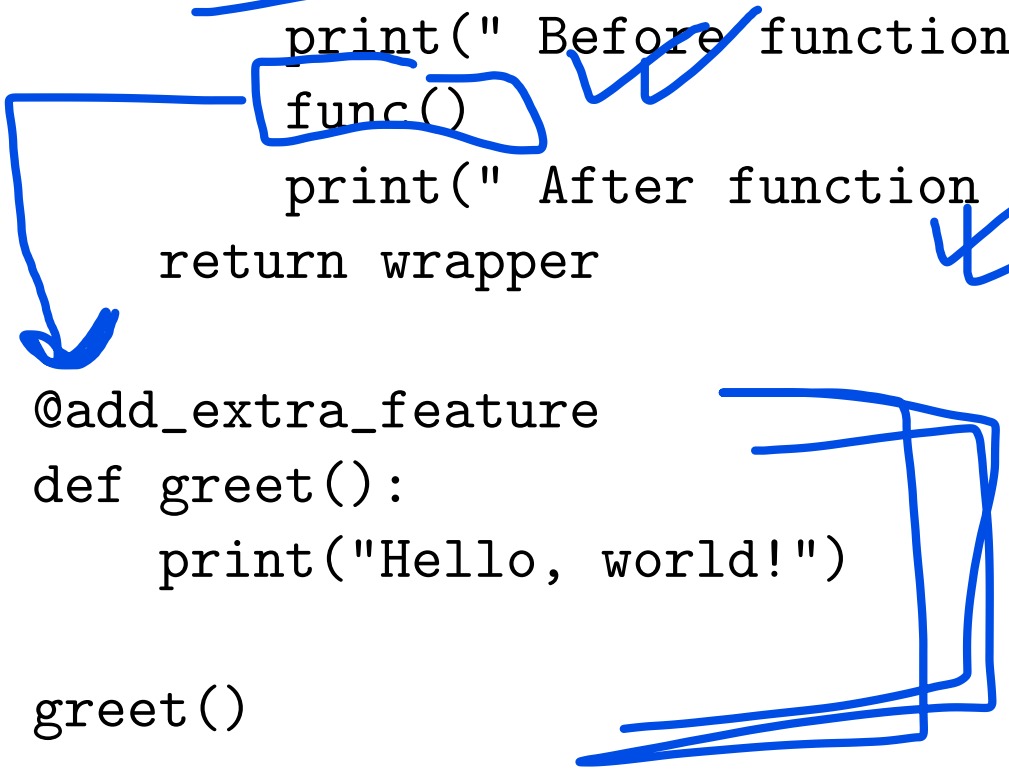
Program: Extending a Function Without Using Decorators

```
def add_extra_feature(func):
    def wrapper():
        print(" Before function runs")
        func()
        print(" After function runs")
    return wrapper

def greet():
    print("Hello, world!")

# Manually calling wrapper
extended_greet = add_extra_feature(greet)
extended_greet()
```

Extension of Function With Decorators

Program: Extending a Function Using a Decorator

```
def add_extra_feature(func):
    def wrapper():
        print(" Before function runs")
        func()
        print(" After function runs")
    return wrapper

@add_extra_feature
def greet():
    print("Hello, world!")

greet()
```



Decorators for Two Functions

Concept: A single decorator can add common behavior to **multiple functions**. This allows code reuse and avoids repetition.

In this example, the same decorator `show_info` is applied to two different functions : `greet()` and `welcome()`.

Decorator Definition

Decorator: Adds extra functionality to a function

```
def show_info(func):
    def wrapper():
        print(">>> Function is starting...")
        func()
        print(">>> Function finished.\n")
    return wrapper
```

Using Same Decorator for Two Functions

```
@show_info
def greet():
    print("Hello from greet() function!")

@show_info
def welcome():
    print("Welcome from welcome() function!")

greet()
welcome()
```

Use Cases of Decorators

Common Use Cases of Python Decorators:

- Logging
- Authentication / Authorization
- Performance measurement (execution time)
- Input validation
- Caching / memoization
- Rate limiting
- Access control
- Debugging and tracing

Decorator Use Case 1: Logging

Logging (Most Common Use Case)

Used to track when a function runs, its inputs, and outputs.

```
def log(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} finished")
        return result
    return wrapper

@log
def process_data():
    print("Processing...")
```

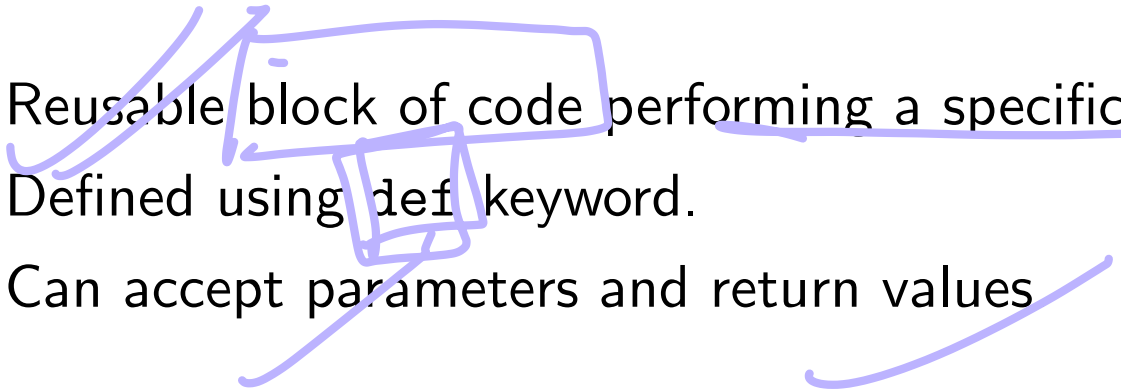
Decorator Use Case 2: Authentication / Authorization

Authentication / Authorization

Used in websites (Flask, Django) to check if a user is logged in.

```
def login_required(func):
    def wrapper():
        print("Checking login...")
        func()
    return wrapper

@login_required
def dashboard():
    print("Welcome to dashboard")
```

Functions in Python



- Reusable block of code performing a specific task.
- Defined using `def` keyword.
- Can accept parameters and return values

Function Examples

Example 1: Simple Function

```
def greet(name):
    return f"Hello, {name}!"

print(greet("Alice"))
```

Example 2: Function with Multiple Arguments

```
def add(a, b):
    return a + b

print(add(5, 3))
```

Lambda Expressions in Python

- Anonymous function defined using `lambda`.
- Can take any number of arguments but only one expression.
- Expression is automatically returned.

Lambda Function Examples

Example 1: Basic Lambda

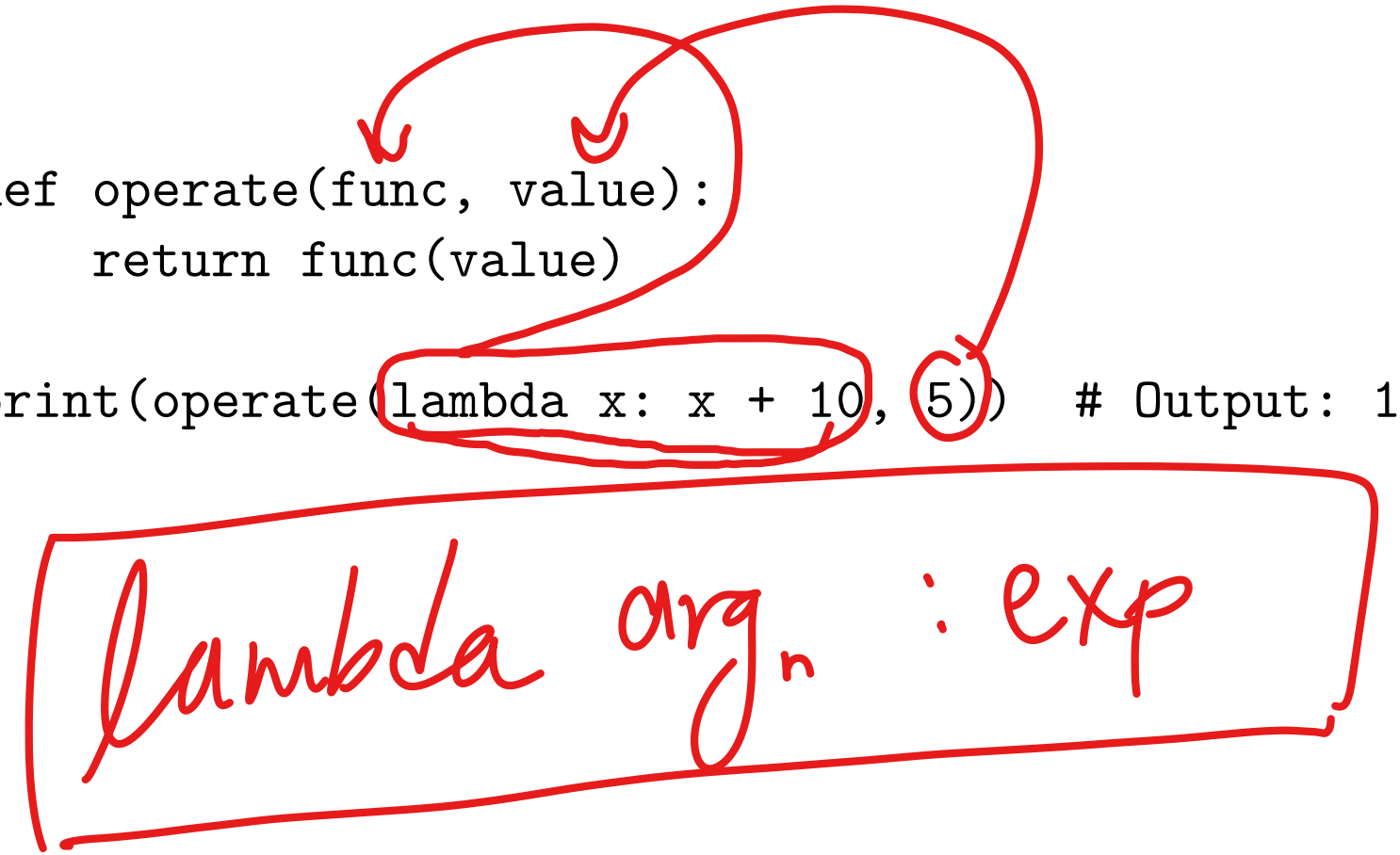
```
square = lambda x: x**2
print(square(5))  # Output: 25
```

Example 2: Lambda with Multiple Arguments

```
multiply = lambda a, b: a * b
print(multiply(4, 3))  # Output: 12
```

Lambda as Function Argument

```
def operate(func, value):
    return func(value)

print(operate(lambda x: x + 10, 5))  # Output: 15
```



Function vs Lambda

Feature	Regular Function	Lambda Function
Keyword	def	lambda
Name	Usually named	Anonymous (can assign to variable)
Return	Uses return	Implicit return
Complexity	Multi-line	Single expression

Lambda in map() Example

```
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared)  # Output: [1, 4, 9, 16]
```

Lambda with filter()

Using lambda to filter a list:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Filter even numbers
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)  # Output: [2, 4, 6, 8, 10]
```

Lambda with sorted()

Using lambda to sort a list:

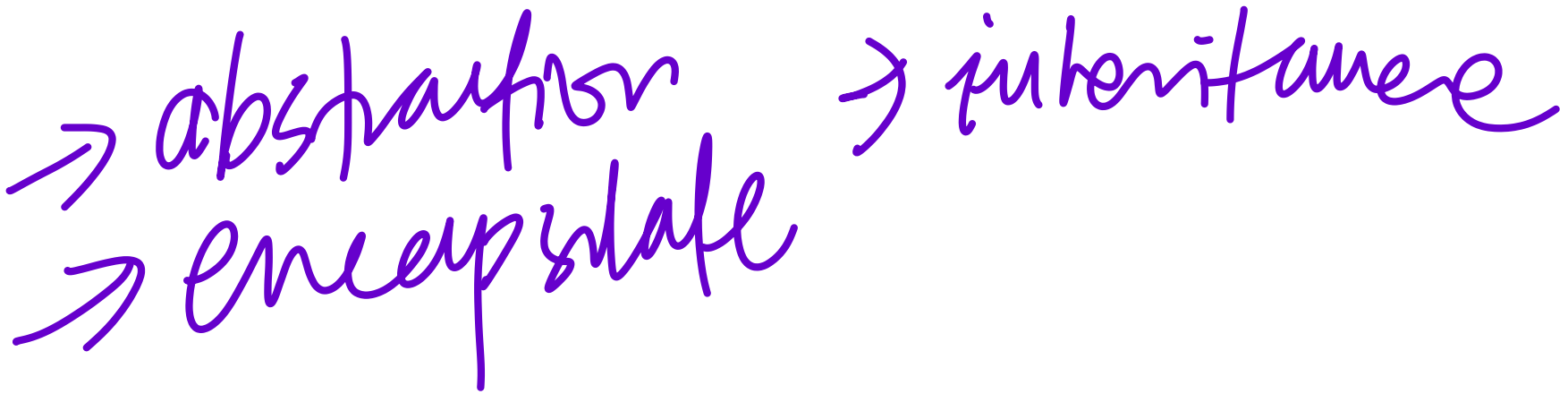
```
words = ["apple", "banana", "kiwi", "mango"]

# Sort words by length
sorted_words = sorted(words, key=lambda w: len(w))
print(sorted_words)  # Output: ['kiwi', 'mango', 'apple', 'ban

numbers = [5, 2, 9, 1, 7]

# Sort numbers in descending order
sorted_numbers = sorted(numbers, key=lambda x: -x)
print(sorted_numbers)  # Output: [9, 7, 5, 2, 1]
```

Introduction to OOP in Python

- Object-Oriented Programming (OOP) organizes code around **objects**.
- Objects have **attributes** (data) and **methods** (functions).
- Key concepts:
  - Class and Object
  - Encapsulation
  - Inheritance
  - Polymorphism
  - Abstraction

Why OOP is Useful for Data Science

- Structure complex workflows: preprocessing, modeling, visualization.
- Reuse code via inheritance (e.g., multiple ML models sharing steps).
- Encapsulation prevents accidental data changes.
- Makes large projects maintainable and collaborative.



Encapsulation: Dataset Handling

```
class Dataset:
    def __init__(self, data):
        self.__data = data  # private attribute

    def get_data(self):
        return self.__data

    def summary(self):
        print(f"Number of records: {len(self.__data)}")
        print(f"Columns: {list(self.__data[0].keys())}")

data = [{"age":25,"income":50000}, {"age":30,"income":60000}]
dataset = Dataset(data)
dataset.summary()
```

Inheritance: Machine Learning Models

```
class MLModel:
    def train(self):
        print("Training the model...")
    def evaluate(self):
        print("Evaluating the model...")

class LinearRegressionModel(MLModel):
    def train(self):
        print("Training Linear Regression model...")

model = LinearRegressionModel()
model.train()     # Training Linear Regression model...
model.evaluate()  # Evaluating the model...
```

Polymorphism: Data Visualization

```
class Plot:
    def draw(self):
        pass

class ScatterPlot(Plot):
    def draw(self):
        print("Drawing Scatter Plot...")

class LinePlot(Plot):
    def draw(self):
        print("Drawing Line Plot...")

plots = [ScatterPlot(), LinePlot()]
for plot in plots:
    plot.draw()
```

Modular OOP for Advanced Projects

- Pipeline classes: combine preprocessing, modeling, evaluation.
- Reusable utility classes: logging, caching, data loading.
- Extensible architecture: add new ML models or visualizations without changing existing code.
- Promotes maintainability, collaboration, and scalability in projects.

Summary

- OOP organizes code around objects, attributes, and methods.
- Encapsulation ensures data safety.
- Inheritance promotes code reuse.
- Polymorphism allows flexibility.
- Widely used in data science frameworks:
  - `scikit-learn` (ML models)
  - `pandas` (DataFrame)
  - `matplotlib`/`seaborn` (plots)

Access Modifiers in Python

- Python uses naming conventions to indicate member accessibility.
- Three types of members:
  1. **Public:** Accessible everywhere.
  2. **Protected:** Intended for internal use in class and subclasses.
  3. **Private:** Intended for internal use only; name mangled.
- Note: Python does not enforce strict access control; conventions are trusted.

Examples of Public, Protected, and Private Members

```
class Person:
    def __init__(self, name, age):
        self.name = name        # Public
        self._age = age         # Protected
        self.__salary = 50000   # Private

p = Person("Alice", 30)
print(p.name)         # Public, accessible
print(p._age)
# Protected, accessible but by convention not used outside
# print(p.__salary)  # Private,  AttributeError
print(p._Person__salary)
# Access via name mangling (not recommended)
```

Access Modifiers Summary

Access Level	**Notation**	**Accessibility**
Public	name	Everywhere
Protected	_name	Inside class & subclasses (convention)
Private	__name	Inside class only (name mangled)

Introduction to Exception Handling

- Exceptions occur when Python encounters an error during execution.
- Exception handling allows you to gracefully manage errors without crashing the program.
- Common built-in exceptions:
  - **ZeroDivisionError** – division by zero
  - **ValueError** – invalid value
  - **TypeError** – unsupported operation between types
  - **IndexError** – invalid list index
  - **KeyError** – invalid dictionary key
  - **FileNotFoundError** – missing file

Exception Handling Blocks in Python

- Python uses blocks to organize exception handling.
- Key blocks:
  - **try:** Contains code that might raise an exception.
  - **except:** Handles exceptions raised in the try block.
  - **else:** Executes if no exception occurs.
  - **finally:** Executes regardless of exceptions (for cleanup).

try and except Blocks

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input!")
```

else and finally Blocks

```
# else block
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("Invalid input!")
else:
    print("You entered:", num)

# finally block
try:
    f = open("data.txt", "r")
except FileNotFoundError:
    print("File not found!")
finally:
    print("Execution finished.")
```

Exception Handling Blocks Summary

Block	Purpose
try	Wrap code that may raise exceptions
except	Handle exceptions that occur
else	Execute if no exception occurs
finally	Execute regardless of exceptions (cleanup)

What is Debugging?

Debugging is the process of identifying, analyzing, and fixing errors or bugs in a program. Python provides several tools and techniques to make this easier.

Types of Errors in Python

- **Syntax Errors** – Mistakes in the code structure.

Example

```
print("Hello World"
```

- **Runtime Errors** – Errors that occur when the program is running.

Example

```
x = 5 / 0
```

- **Logical Errors** – Code runs but produces incorrect results.

Example

```
total = 2 + 2 * 3
```

Debugging Techniques

Common techniques for debugging in Python:

- Using `print()` statements
- Using Python's built-in `pdb` module
- Using `try-except` blocks
- Using IDE debuggers
- Using `logging` module

Using print() Statements

The simplest method to check variable values and flow.

Example

```
def add_numbers(a, b):
    print(f"a={a}, b={b}")  # Debug info
    return a + b

result = add_numbers(5, 10)
print(result)
```

- Pros: Easy and quick
- Cons: Clutters code, not suitable for large projects

Using Python's pdb Module

Python's debugger allows step-by-step execution.

Example

```
import pdb

def divide(a, b):
    pdb.set_trace()  # Start debugging here
    return a / b

result = divide(10, 0)
print(result)
```

Common pdb Commands:

- n → Next line
- s → Step into function
- c → Continue execution
- q → Quit debugger
- p variable → Print variable value

Using try-except Blocks

Handle runtime errors gracefully.

Example

```
try:
    x = 10 / 0
except ZeroDivisionError as e:
    print("Error occurred:", e)
```

- Pros: Prevents program crash
- Cons: Does not fix the underlying logic issue

Using IDE Debuggers

Most IDEs like PyCharm, VS Code, Jupyter Notebook have built-in debuggers. Features include:

- Breakpoints
- Step execution
- Variable inspection
- Call stack view

Using Logging Instead of Print

For larger projects, use Python's `logging` module.

Example

```
import logging

logging.basicConfig(level=logging.DEBUG, filename="app.log",
filemode="w",format="%(name)s - %(levelname)s - %(message)s")

def multiply(a, b):
    logging.debug(f"Multiplying {a} and {b}")
    return a * b

result = multiply(5, 4)
```

- Check `app.log` for debug messages
- Logging levels: DEBUG, INFO, WARNING, ERROR, CRITICAL

Output (console):

- DEBUG:root:This is a debug message
- INFO:root:This is an info message
- WARNING:root:This is a warning message
- ERROR:root:This is an error message
- CRITICAL:root:This is a critical message

Logging to a File

For larger projects, logging messages can be saved to a file instead of the console.

Python Code

```
import logging

logging.basicConfig(
    filename="app.log",
    filemode="w",  # Overwrite each run; use "a" to append
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Program started")
logging.warning("This is a warning")
logging.error("This is an error")
```

Notes:

What is a Module?

A **module** in Python is a file containing Python code (functions, classes, variables, or runnable code) that can be imported and used in other Python programs.

- Helps organize code
- Allows code reuse

Example: User-defined module

mymodule.py

```
def greet(name):
    print(f"Hello, name!")
```

```
import mymodule
mymodule.greet("Alice")
```

Types of Modules

- **Built-in Modules** – Provided by Python standard library (e.g., math, os, sys)

Example

```
import math
print(math.sqrt(16))
```

- **User-defined Modules** – Created by programmers

Example

```
import mymodule
print(mymodule.add(5,10))
```

- **Third-party Modules** – Installed via pip (e.g., numpy, pandas, requests)

Importing Modules

- `import module_name` – Import whole module
- `from module_name import function_name` – Import specific function
- `import module_name as alias` – Use a shorter name

Example

```
from math import sqrt
print(sqrt(25))
import numpy as np
arr = np.array([1,2,3])
```

Why Use Modules?

- Organize code into logical sections
- Avoid code duplication
- Reusability across projects
- Easier maintenance and debugging

Example: Creating and Using a Module

calculator.py

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

main.py

```
import calculator

print(calculator.add(10,5))       # Output: 15
print(calculator.subtract(10,5))  # Output: 5
```

What is a Package?

A **package** in Python is a collection of modules organized in a directory.

- Helps structure Python code hierarchically
- Identified by the presence of an `__init__.py` file (can be empty)

Directory structure example:

```
my_package/
    __init__.py
    module1.py
    module2.py
```

Usage:

```
from my_package import module1
module1.function_name()
```

Module vs Package

Feature	Module	Package
Definition	Single .py file	Directory of modules
Purpose	Reuse code in one file	Organize multiple modules hierarchically
Example	calculator.py	my_package containing module1.py, module2.

Creating a Simple Package

Directory structure:

```
math_package/
    __init__.py
    addition.py
    subtraction.py
```

addition.py

```
def add(a, b):
    return a + b
```

subtraction.py

```
def subtract(a, b):
    return a - b
```

Using the Package

main.py

```
from math_package import addition, subtraction

print(addition.add(5, 3))         # Output: 8
print(subtraction.subtract(5, 3)) # Output: 2
```

Importing from Packages

- `import package.module` – Import entire module
- `from package.module import function` – Import specific function
- `import package.module as alias` – Import with alias

Example

```
from math_package.addition import add
print(add(10, 5))  # Output: 15
```

Why Use Packages?

- Organize code in large projects
- Avoid name conflicts
- Enable reusability
- Maintain modular and readable code

Thank You!